

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

backend/src/controllers/auth.controller.js

```
1.  /**
2.   * @file Controlador de autenticación.
3.   * @description Gestiona el registro, inicio de sesión y perfil de l
4.   * @requires jsonwebtoken
5.   * @requires ../models/usuarios_mongoose
6.   */
7.  const jwt = require('jsonwebtoken');
8.  const User = require('../models/usuarios_mongoose');
9.
10. /**
11.  * @function registerUser
12.  * @description Registra un nuevo usuario en la base de datos.
13.  * @param {object} req - Objeto de petición de Express.
14.  * @param {object} res - Objeto de respuesta de Express.
15.  * @returns {Promise<void>}
16.  */
17.  const registerUser = async (req, res) => {
18.    try {
19.      const { email, password, name } = req.body;
20.
21.      // Validación de campos requeridos
22.      if (!email || !password || !name) {
23.        return res.status(400).json({ message: 'Email, password
24.      }
25.
26.      // Validar longitud de nombre
27.      if (name.trim().length < 2) {
28.        return res.status(400).json({ message: 'Name must be at
29.      }
30.
31.      // Validar formato de email
32.      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
33.      if (!emailRegex.test(email)) {
34.        return res.status(400).json({ message: 'Invalid email fo
35.      }
36.
37.      // Validar longitud de contraseña
38.      if (password.length < 8) {
39.        return res.status(400).json({ message: 'Password must be
40.      }
41.
42.      // Verificar si el usuario ya existe
43.      const existingUser = await User.findOne({ email });
44.      if (existingUser) {
45.        return res.status(400).json({ message: 'User with this e
46.      }
47.
48.      // Guardar el usuario en la base de datos
49.      const newUser = await User.create({
50.        email,
51.        password: password, // El hook pre('save') del modelo l
52.        nombre: name,
```

```

53.     });
54.
55.     // Crear y firmar el token JWT
56.     const payload = {
57.         id: newUser.id,
58.         email: newUser.email,
59.         name: newUser.nombre,
60.     };
61.
62.     console.log('    Creando token JWT con payload:', payload);
63.
64.     const token = jwt.sign(
65.         payload,
66.         process.env.JWT_SECRET,
67.         { expiresIn: '1h' }
68.     );
69.
70.     console.log('    Token creado exitosamente en registerUser:',
71.
72.     // Devolver token Y datos del usuario (sin contraseña)
73.     const response = {
74.         message: 'User registered successfully',
75.         token: token,
76.         user: {
77.             id: newUser.id,
78.             email: newUser.email,
79.             nombre: newUser.nombre,
80.             alias: newUser.alias,
81.             createdAt: newUser.createdAt,
82.             updatedAt: newUser.updatedAt
83.         }
84.     };
85.
86.     console.log('    Enviando respuesta de registro con token vál
87.
88.     res.status(201).json(response);
89. } catch (error) {
90.     console.error('    Error en registerUser:', error);
91.     // Manejo específico de errores de validación de Mongoose
92.     if (error.name === 'ValidationError') {
93.         const messages = Object.values(error.errors).map(err =>
94.         return res.status(400).json({
95.             message: 'Validation error',
96.             errors: messages
97.         });
98.     }
99.
100.    // Error de email duplicado (índice único)
101.    if (error.code === 11000) {
102.        return res.status(400).json({
103.            message: 'User with this email already exists'
104.        });
105.    }
106.
107.    // Otros errores del servidor
108.    res.status(500).json({ message: 'Error registering user', er
109.    }
110.    };
111.
112.    /**
113.    * @function loginUser
114.    * @description Autentica a un usuario y le proporciona un token JWT
115.    * @param {object} req - Objeto de petición de Express.
116.    * @param {object} res - Objeto de respuesta de Express.

```

```
117.    * @returns {Promise<void>}
118.    */
119.    const loginUser = async (req, res) => {
120.      try {
121.        const { email, password } = req.body;
122.
123.        // Validacion
124.        if (!email || !password) {
125.          return res.status(400).json({ message: 'Email and password required' });
126.        }
127.
128.        console.log('  Intentando login para:', email);
129.
130.        const user = await User.findOne({ email });
131.        if (!user) {
132.          console.log('  Usuario no encontrado:', email);
133.          return res.status(401).json({ message: 'Invalid credentials' });
134.        }
135.
136.        const isMatch = await user.compararPassword(password);
137.        if (!isMatch) {
138.          console.log('  Contraseña incorrecta para:', email);
139.          return res.status(401).json({ message: 'Invalid credentials' });
140.        }
141.
142.        console.log('  Credenciales válidas para:', email);
143.
144.        // Crear y firmar el token JWT
145.        const payload = {
146.          id: user.id,
147.          email: user.email,
148.          name: user.nombre,
149.        };
150.
151.        const token = jwt.sign(
152.          payload,
153.          process.env.JWT_SECRET,
154.          { expiresIn: '1h' }
155.        );
156.
157.        console.log('  Token generado para login:', token.substring(0, 10));
158.
159.        // Devolver token Y datos del usuario (sin contraseña)
160.        const response = {
161.          message: 'User logged in successfully',
162.          token: token,
163.          user: {
164.            id: user.id,
165.            email: user.email,
166.            nombre: user.nombre,
167.            alias: user.alias,
168.            createdAt: user.createdAt,
169.            updatedAt: user.updatedAt
170.          }
171.        };
172.
173.        console.log('  Enviando respuesta de login');
174.
175.        res.status(200).json(response);
176.      } catch (error) {
177.        console.error('  Error en loginUser:', error);
178.        res.status(500).json({ message: 'Error logging in', error: error.message });
179.      }
180.    };
```

```

181.
182. /**
183.  * @function getProfile
184.  * @description Obtiene el perfil del usuario autenticado.
185.  * @param {object} req - Objeto de petición de Express, con la infor
186.  * @param {object} res - Objeto de respuesta de Express.
187.  * @returns {Promise<void>}
188.  */
189. const getProfile = async (req, res) => {
190.   try {
191.     console.log('  getProfile llamado para usuario:', req.user?
192.
193.     // req.user viene del middleware de autenticación
194.     const user = await User.findById(req.user.id).select('-passw
195.
196.     if (!user) {
197.       console.log('  Usuario no encontrado con ID:', req.user
198.       return res.status(404).json({ message: 'User not found'
199.     }
200.
201.     console.log('  Perfil obtenido para:', user.email);
202.
203.     res.status(200).json({
204.       user: {
205.         id: user.id,
206.         email: user.email,
207.         nombre: user.nombre,
208.         alias: user.alias,
209.         createdAt: user.createdAt,
210.         updatedAt: user.updatedAt
211.       }
212.     });
213.   } catch (error) {
214.     console.error('  Error en getProfile:', error);
215.     res.status(500).json({ message: 'Error fetching profile', er
216.   }
217. };
218.
219. module.exports = {
220.   registerUser,
221.   loginUser,
222.   getProfile,
223. };
224.

```